

Refer to this folder:

"C:\Users\Vincent_Pereira\Projects\Algo_Trading_Projects\Vincent Pereira\IBKR Paper Trading System"

Build a Web Application based Dashboard which will run these 3 strategies and monitor the scrips selected and show which trades have been placed and want is the current position.

Market Watch Dashboard Application:

Use Folder: "C:\Users\Vincent_Pereira\Projects\Algo_Trading_Projects\Vincent Pereira\IBKR Paper Trading System"

Objective:

Develop a Market Watch Dashboard for stocks that provides real-time data, technical analysis indicators, and interactive charts, leveraging my Interactive Brokers (IB) Paper Trading account for data and testing.

The dashboard will be built as a web application using Python with the Plotly Dash framework or Node.js or whatever you think is the best. You will leverage `ib_insync` to integrate with external data sources and tools, including the Interactive Brokers API, during development.

Make the application beautiful and modern.

Following are guidelines for you to work on, but if you feel there is a better way of doing it, you may use your discretion and execute accordingly.

Design the Dashboard Features

- **User Interface:**

- Create an interactive web interface using Plotly Dash, or whatever is better, where users can:
 - Select stocks or indices by ticker symbol (e.g., AAPL, TSLA) via a dropdown or input field.
 - View real-time stock prices (bid, ask, last price, volume, etc.) fetched from the IB Live Trading account in a table or ticker format.
 - Display interactive charts (e.g., candlestick or line charts) with zoom and pan capabilities, using data from IB or the fallback API.
 - Overlay customisable technical indicators (e.g., SMA, EMA, RSI, MACD, MFI, Gann Angles, Gann Fan, Gann Box, Support and Resistance, etc.) with adjustable parameters (timeframes, colors).
 - Show portfolio summaries from both IB accounts (e.g., positions, account value, unrealised P&L).
 - Include a simulated trading panel connected to the Paper Trading account, allowing users to place virtual buy/sell orders for testing.
 - Add a watchlist feature to save and monitor favourite stocks.
- There should also be an option to select a Scrip, and the application should provide the complete Technical Analysis for that Scrip. As per the technical analysis, it should tell the outlook for that scrip for that day (Intra Day) as well as where the stock is likely headed in the next 5 days (one week), whether Strong Up Trend, Up Trend, Side-ways, Down Trend or Strong Down Trend, etc. It should also mention about the position of the Daily Time Frame indicators like RSI(14), MFI(14), MACD, SMAs, EMAs, etc.
- Market Watchlist based on:
 - Highest Volatility Scrips
 - Highest Trading Volume of Scrips
- Include a Trading Strategy using Support and Resistance levels for that Scrip.
- It should also provide the latest news articles for that Scrip and if there are any main events to be held, like AGM, Dividends, etc. for that scrip in the next 10 days.

- Ensure the design is responsive for various screen sizes, including for smart phones, iPhone as well as Android.
- **Data Handling:**
 - Fetch stock data (e.g., open, high, low, close, volume, bid, ask, ltp, etc.) from the IB Live Trading account using `ib_insync`.
 - Retrieve portfolio and order data from both Live and Paper Trading accounts for display.
 - Compute technical indicators `pandas_ta` locally.
 - Implement automatic data updates (e.g., polling IB every 5-10 seconds for real-time quotes) with efficient caching to respect IB API rate limits.
 - For simulated trades, send orders to the Paper Trading account only, ensuring no live trades are executed unless explicitly enabled.

The simulated trading feature should include advanced order types (e.g., limit, stop) and not just basic market orders.

Project Setup: Generate the initial project structure, including files like `app.py` for the Dash application, and install dependencies via terminal commands.

Code Generation: Write code for:

- Connecting to the IB API using `ib_insync` with my Live and Paper Trading API keys.
- Fetching real-time stock data and portfolio details from IB accounts.
- Computing technical indicators and rendering charts in Dash.
- Implementing a simulated trading interface for the Paper Trading account.
- Example requests: “Generate a Python function to connect to Interactive Brokers using `ib_insync` with an API key” or “Create a Dash layout with a candlestick chart and portfolio table.”

Debugging and Testing: Debug errors (e.g., IB API connection timeouts, data mismatches) and test functions with sample data from the IB Paper Trading account.

Terminal Execution: Run the Dash development server (`python app.py`) and verify the dashboard loads in a browser.

Example Cline commands:

- “Plan a Python script for a stock dashboard with Plotly Dash and Interactive Brokers API.”
- “Act: Write code to fetch real-time AAPL data from IB and plot it as a candlestick chart.”

Implementation Details

- **Backend:**
 - Use Python with Plotly Dash to serve the dashboard. Use `ib_insync` to handle IB API interactions for both accounts.
 - Write functions to:
 - Establish connections to IB TWS/IB Gateway for Live and Paper Trading accounts using your API keys.
 - Fetch real-time and historical stock data from the Live Trading account.
 - Retrieve portfolio and order data from both accounts.
 - Send simulated orders to the Paper Trading account for testing.
 - Compute technical indicators via `pandas_ta`.
- **Frontend:**
 - **React.js** for a responsive, interactive, and modern web interface,
 - **Plotly.js** for interactive visualizations (e.g., candlestick charts, equity curves), and
 - **Tailwind CSS** for modern, clean styling, making the page look good and responsive across different devices.
 - **JavaScript** for dynamic behaviour and interactivity, allowing elements to change, data to be fetched without reloading the page, and complex user interactions.
 - Use Plotly Dash components (`dcc.Graph`, `html.Div`, `dash_table`) to build the UI.

- Render interactive charts with Plotly's candlestick or line chart types, overlaying indicators as traces.
 - Display portfolio data in a table, separating Live and Paper Trading accounts.
 - Add a form for simulated trades (e.g., ticker, quantity, order type) linked to the Paper Trading account.
 - Implement Dash callbacks to update the UI dynamically based on user inputs (e.g., changing stocks, submitting trades).
- **Extensibility:**
 - Design the code to allow swapping data sources.
 - Ensure the simulated trading feature is isolated to the Paper Trading account to prevent accidental live trades.

Deployment and Final Touches

- Test the dashboard locally to ensure:
 - Real-time data updates from the IB Live Trading account.
 - Accurate portfolio displays for both accounts.
 - Correct calculation and visualisation of technical indicators.
 - Functional simulated trading via the Paper Trading account.
- Deploy the web application on a local server, ensuring it remains accessible via a browser.
- Optimise performance by respecting IB API rate limits (e.g., caching data for 5-10 seconds) and ensuring smooth UI updates.
- Secure the IB API keys in the deployed application (e.g., using environment variables on the hosting platform).
- Document the code and provide usage instructions, including how to configure IB API keys and switch between Live and Paper Trading modes.

Notes:

- Use the IB Paper Trading account extensively during development and testing to avoid unintended live trades.
- Ensure the dashboard clearly distinguishes between Live and Paper Trading data to prevent confusion.
- Focus on usability: the dashboard should be intuitive for monitoring stocks, analysing trends, and testing trades safely.
- Handle IB API rate limits and connection stability carefully, as IB's API can be sensitive to frequent requests or network issues.

Later, I would like to include my trading strategies including position sizing and money management rules. For now, develop a trading strategy using Support and Resistance with Entry, Exit, Stop Loss and Targets, etc.

Let me know if you need any information or clarification.

IB API Details for Paper Trading Account:

- Host: 127.0.0.1
- Port: 7497 (Paper Trading port)
- Client ID: 0
- IBKR Account No.: DUK221396
- The `ibapi` folder is located at the following address:
"C:/TWS API/source/pythonclient"